

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**“БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. Шухова”(БГТУ им. В. Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и
автоматизированных систем

Лабораторная работа №1

по дисциплине: Тестирование программных систем

тема: «Составление тест-кейсов для готового кода.

Качество ПО и место тестирования в жизненном
цикле»

Выполнил: ст. группы ВТ-231

Шпак Никита Алексеевич

Проверил:

Доденко Олег Павлович

Белгород, 2026 г.

Лабораторная работа №1 «Составление тест-кейсов для готового кода.
Качество ПО и место тестирования в жизненном цикле»

Цель работы: сформировать системное представление о качестве ПО и роли тестирования в жизненном цикле; приобрести практические навыки проектирования тест-кейсов и написания автотестов на основе готового кода.

Задачи:

1. Проанализировать готовый модуль и выделить бизнес-правила, входные и выходные данные, ограничения.
2. Составить набор тест-кейсов: позитивных, негативных, граничных, покрывающих эквивалентные классы.
3. Оформить тест-кейсы по стандартным атрибутам.
4. Реализовать автотесты на Python + pytest.
5. Запустить тесты и зафиксировать результаты.
6. Сделать вывод о связи тестирования, качества ПО и жизненного цикла.

Задания:

1. Изучить исходный код и выписать все бизнес-правила.
2. Определить:
 - входные параметры;
 - выходной параметр;
 - допустимые и недопустимые значения;
 - граничные значения.
3. Составить не менее 10 тест-кейсов:
 - позитивные;
 - негативные;
 - граничные;
 - проверяющие скидку для premium-пользователей;
 - проверяющие бесплатную доставку.
4. Написать автотесты на pytest.
5. Запустить тесты командой `pytest -v`.
6. Сделать вывод: какие ветви кода покрыты, какие риски остались, почему тестирование важно для качества.

Ход выполнения лабораторной работы

Задание 1:

В качестве своего кода разработал модуль расчёта стоимости аренды электросамоката `rental.py`:

```

def calculate_rental_cost(minutes: int, distance_km: float,
is_student: bool) -> float:
    """
    Расчёт стоимости аренды электросамоката.

    Бизнес-правила:
    1. Если minutes < 0 или distance_km < 0 — ValueError.
    2. Если minutes == 0 — поездка не состоялась, стоимость 0.
    3. Иначе:
        - первая часть поездки до 30 минут: 100 (базовый тариф)
        - 30 < minutes <= 60: 100 + (minutes - 30) * 5
        - minutes > 60: 250 + (minutes - 60) * 8
    4. За каждый км свыше 10 добавляется 20 (доплата за износ).
    5. Для is_student=True применяется скидка 30% на итоговую
    стоимость.
    """
    if minutes < 0:
        raise ValueError("minutes must be >= 0")
    if distance_km < 0:
        raise ValueError("distance_km must be >= 0")

    if minutes == 0:
        base = 0.0
    elif minutes <= 30:
        base = 100.0
    elif minutes <= 60:
        base = 100.0 + (minutes - 30) * 5.0
    else:
        base = 250.0 + (minutes - 60) * 8.0

    if distance_km > 10:
        base += (distance_km - 10) * 20.0

    if is_student:
        base *= 0.7

    return base

```

Бизнес-правила:

№	Бизнес-правило	Реализация в коде	Тестовые границы
BR1	Если minutes < 0 — ValueError	if minutes < 0: raise ValueError	-1 / 0
BR2	Если distance_km < 0 — ValueError	if distance_km < 0: raise ValueError	-1 / 0
BR3	Если minutes == 0 — поездка не состоялась, стоимость равна 0	ветка if minutes == 0: base = 0.0	0 / 1

BR4	При $0 < \text{minutes} \leq 30$ действует базовый тариф — 100	ветка <code>elif minutes <= 30:</code> <code>base = 100.0</code>	1 / 30
BR5	При $30 < \text{minutes} \leq 60$ — $100 + (\text{minutes} - 30) * 5$	ветка <code>elif minutes <= 60</code>	31 / 60
BR6	При $\text{minutes} > 60$ — $250 + (\text{minutes} - 60) * 8$	ветка <code>else</code>	61
BR7	За каждый км свыше 10 добавляется 20 (доплата за износ)	<code>if distance_km > 10: base</code> <code>+= (distance_km - 10) * 20</code>	10 / 10.01 / 15
BR8	При <code>is_student = True</code> итоговая стоимость умножается на 0.7 (скидка 30%)	<code>if is_student: base *= 0.7</code>	True / False

В модуле `rental.py` выделил 8 бизнес-правил, которые делятся на три группы:

- ограничения** (BR1, BR2) — валидация входных данных;
- вычисления** (BR3–BR7) — тарифная сетка и доплата;
- модификаторы условий** (BR8) — скидка студента.

Задание 2:

Входные параметры:

Функция `calculate_rental_cost(minutes: int, distance_km: float, is_student: bool)`

-> float принимает три входных параметра:

- `minutes` – int – длительность поездки в минутах – определяет базовый тариф (BR3–BR6)
- `distance_km` – float – пройденное расстояние в километрах – определяет доплату за износ (BR7)
- `is_student` – bool – студенческая скидка – модификатор итоговой суммы (BR8)

Выходной параметр:

- возвращаемое значение – float – Итоговая стоимость аренды электросамоката

Допустимые значения и недопустимые значения:

`minutes (int)`

- Допустимые: 0 → стоимость 0 (поездки не было)
- Допустимые: 1 ... 30 → базовый тариф 100
- Допустимые: 31 ... 60 → $100 + (m-30) \cdot 5$

- Допустимые: $> 60 \rightarrow 250 + (m-60) \cdot 8$
- Недопустимые: $< 0 (-1, -100...) \rightarrow \text{ValueError}$

distance_km (float)

- Допустимые: $0 \dots 10 \rightarrow$ без доплаты за расстояние
- Допустимые: $> 10 \rightarrow + (km-10) \cdot 20$ за каждый км сверх 10
- Недопустимые: $< 0 (-0.5, -1...) \rightarrow \text{ValueError}$

is_student (bool)

- Допустимые: False $\rightarrow$ скидка не применяется
- Допустимые: True $\rightarrow$ итог умножается на 0.7

Граничные значения:

Границы определяются точками переключения логики — там, где меняется формула ($<$, $<=$, $>$).

По времени (minutes):

Граница	Значение «до»	Значение «на»	Значение «после»	Что меняется
Начало допустимого	—	0	1	0 (не было) $\rightarrow$ базовый тариф 100
Конец базового тарифа	29	30	31	100 $\rightarrow$ переменная формула
Конец второго тарифа	59	60	61	$100+(m-30) \cdot 5 \rightarrow 250+(m-60) \cdot 8$

Проверяем: 0 / 30 / 31 / 60 / 61 (плюс «чуть меньше/больше» — 29, 59).

По расстоянию (distance_km):

Граница	Значение «до»	Значение «на»	Значение «после»	Что меняется
Порог доплаты	9.99	10	10.01	без доплаты $\rightarrow + (km-10) \cdot 20$

Проверяем: 10 (без доплаты) и 10.01 / 15 (с доплатой).

По скидке (is_student):

Граница	False	True	Что меняется
Переключение скидки	без скидки	$\times 0.7$	итоговая сумма

Задание 3:

Перечень тест-кейсов:

Позитивные тест-кейсы (TC01–TC04):

TC01

ID	TC01
Название	Минимальная поездка (базовый тариф)
Приоритет	High

Тип	Позитивный
Входные данные	minutes=1, distance_km=5, is_student=False
Шаги	1. Вызвать calculate_rental_cost(1, 5, False)
Ожидаемый результат	100.0
Расчёт	$1 \leq 30 \rightarrow$ база 100; км $5 \leq 10 \rightarrow$ без доплаты
Бизнес-правило	BR4
Автотест	test_minimal_ride
Статус	Passed

TC02

ID	TC02
Название	Середина базового тарифа
Приоритет	Medium
Тип	Позитивный
Входные данные	minutes=15, distance_km=5, is_student=False
Шаги	1. Вызвать calculate_rental_cost(15, 5, False)
Ожидаемый результат	100.0
Расчёт	$15 \leq 30 \rightarrow$ база 100
Бизнес-правило	BR4
Автотест	test_base_tariff_middle
Статус	Passed

TC03

ID	TC03
Название	Второй тариф, середина (45 мин)
Приоритет	Medium
Тип	Позитивный
Входные данные	minutes=45, distance_km=5, is_student=False
Шаги	1. Вызвать calculate_rental_cost(45, 5, False)
Ожидаемый результат	175.0
Расчёт	$30 < 45 \leq 60 \rightarrow 100 + 15 \cdot 5 = 175$
Бизнес-правило	BR5
Автотест	test_second_tariff_middle
Статус	Passed

TC04

ID	TC04
Название	Доплата за расстояние (15 км)
Приоритет	High

Тип	Позитивный
Входные данные	minutes=30, distance_km=15, is_student=False
Шаги	1. Вызвать calculate_rental_cost(30, 15, False)
Ожидаемый результат	200.0
Расчёт	база 100; км 15 > 10 → + (15-10)·20 = 100; итого 200
Бизнес-правило	BR7
Автотест	test_distance_surcharge
Статус	Passed

Граничные тест-кейсы (TC05–TC09):

TC05

ID	TC05
Название	Ровно 30 минут (верхняя граница базового тарифа)
Приоритет	High
Тип	Граничный
Входные данные	minutes=30, distance_km=5, is_student=False
Шаги	1. Вызвать calculate_rental_cost(30, 5, False)
Ожидаемый результат	100.0
Расчёт	$30 \leq 30 \rightarrow$ база 100
Бизнес-правило	BR4
Автотест	test_base_tariff_upper_bound
Статус	Passed

TC06

ID	TC06
Название	31 минута (начало второго тарифа)
Приоритет	High
Тип	Граничный
Входные данные	minutes=31, distance_km=5, is_student=False
Шаги	1. Вызвать calculate_rental_cost(31, 5, False)
Ожидаемый результат	105.0
Расчёт	$30 < 31 \leq 60 \rightarrow 100 + 1 \cdot 5 = 105$
Бизнес-правило	BR5
Автотест	test_second_tariff_start
Статус	Passed

TC07

ID	TC07
Название	Ровно 60 минут (верхняя граница второго тарифа)

Приоритет	High
Тип	Граничный
Входные данные	minutes=60, distance_km=5, is_student=False
Шаги	1. Вызвать calculate_rental_cost(60, 5, False)
Ожидаемый результат	250.0
Расчёт	$30 < 60 \leq 60 \rightarrow 100 + 30 \cdot 5 = 250$
Бизнес-правило	BR5
Автотест	test_second_tariff_upper_bound
Статус	Passed

TC08

ID	TC08
Название	61 минута (начало третьего тарифа)
Приоритет	High
Тип	Граничный
Входные данные	minutes=61, distance_km=5, is_student=False
Шаги	1. Вызвать calculate_rental_cost(61, 5, False)
Ожидаемый результат	258.0
Расчёт	$61 > 60 \rightarrow 250 + 1 \cdot 8 = 258$
Бизнес-правило	BR6
Автотест	test_third_tariff_start
Статус	Passed

TC09

ID	TC09
Название	Ровно 10 км (порог доплаты, доплаты нет)
Приоритет	High
Тип	Граничный
Входные данные	minutes=30, distance_km=10, is_student=False
Шаги	1. Вызвать calculate_rental_cost(30, 10, False)
Ожидаемый результат	100.0
Расчёт	база 100; км 10 не $> 10 \rightarrow$ без доплаты
Бизнес-правило	BR7
Автотест	test_no_distance_surcharge_at_10
Статус	Passed

Негативные тест-кейсы (TC10–TC11):

TC10

ID	TC10
----	------

Название	Отрицательное время → ошибка
Приоритет	High
Тип	Негативный
Входные данные	minutes=-1, distance_km=5, is_student=False
Шаги	1. Вызвать calculate_rental_cost(-1, 5, False)
Ожидаемый результат	Возбуждается ValueError
Бизнес-правило	BR1
Автотест	test_negative_minutes
Статус	Passed

TC11

ID	TC11
Название	Отрицательное расстояние → ошибка
Приоритет	High
Тип	Негативный
Входные данные	minutes=10, distance_km=-1, is_student=False
Шаги	1. Вызвать calculate_rental_cost(10, -1, False)
Ожидаемый результат	Возбуждается ValueError
Бизнес-правило	BR2
Автотест	test_negative_distance
Статус	Passed

Скидка студента (TC12–TC13):

TC12

ID	TC12
Название	Скидка студента 30% (базовая)
Приоритет	High
Тип	Позитивный (скидка)
Входные данные	minutes=30, distance_km=5, is_student=True
Шаги	1. Вызвать calculate_rental_cost(30, 5, True)
Ожидаемый результат	70.0
Расчёт	база 100; скидка → $100 \cdot 0.7 = 70$
Бизнес-правило	BR8
Автотест	test_student_discount
Статус	Passed

TC13

ID	TC13
Название	Скидка студента + доплата за километры

Приоритет	High
Тип	Позитивный (скидка)
Входные данные	minutes=61, distance_km=15, is_student=True
Шаги	1. Вызвать calculate_rental_cost(61, 15, True)
Ожидаемый результат	250.6
Расчёт	база $250 + 1 \cdot 8 = 258$; км $+ (15 - 10) \cdot 20 = 100$; итого 358; скидка $358 \cdot 0.7 = 250.6$
Бизнес-правило	BR7 + BR8
Автотест	test_student_discount_with_surcharge (использует pytest.approx)
Статус	Passed

Бесплатная поездка (TC14–TC15):

TC14

ID	TC14
Название	Нулевая поездка — стоимость 0
Приоритет	High
Тип	Граничный / Особый случай
Входные данные	minutes=0, distance_km=5, is_student=False
Шаги	1. Вызвать calculate_rental_cost(0, 5, False)
Ожидаемый результат	0.0
Расчёт	minutes == 0 → 0.0
Бизнес-правило	BR3
Автотест	test_zero_minutes
Статус	Passed

TC15

ID	TC15
Название	Нулевая поездка со скидкой — всё равно 0
Приоритет	Medium
Тип	Граничный / Особый случай
Входные данные	minutes=0, distance_km=5, is_student=True
Шаги	1. Вызвать calculate_rental_cost(0, 5, True)
Ожидаемый результат	0.0
Расчёт	minutes == 0 → 0.0; скидка $0 \cdot 0.7 = 0$
Бизнес-правило	BR3 + BR8
Автотест	test_zero_minutes_with_student
Статус	Passed

Задание 4:

Написал автотесты на pytest.

Код автотестов test_rental.py:

```
# test_rental.py
import pytest
from rental import calculate_rental_cost

# Позитивные тесты (TC01 - TC04)

def test_minimal Ride():
    """TC01: минимальная поездка - базовый тариф."""
    assert calculate_rental_cost(1, 5, False) == 100.0

def test_base_tariff_middle():
    """TC02: середина базового тарифа."""
    assert calculate_rental_cost(15, 5, False) == 100.0

def test_second_tariff_middle():
    """TC03: второй тариф, середина (45 мин)."""
    assert calculate_rental_cost(45, 5, False) == 175.0

def test_distance_surcharge():
    """TC04: доплата за расстояние (15 км)."""
    assert calculate_rental_cost(30, 15, False) == 200.0

# Граничные тесты (TC05 - TC09)

def test_base_tariff_upper_bound():
    """TC05: ровно 30 минут - верхняя граница базового тарифа."""
    assert calculate_rental_cost(30, 5, False) == 100.0

def test_second_tariff_start():
    """TC06: 31 минута - начало второго тарифа."""
    assert calculate_rental_cost(31, 5, False) == 105.0

def test_second_tariff_upper_bound():
    """TC07: ровно 60 минут - верхняя граница второго тарифа."""
    assert calculate_rental_cost(60, 5, False) == 250.0

def test_third_tariff_start():
    """TC08: 61 минута - начало третьего тарифа."""
    assert calculate_rental_cost(61, 5, False) == 258.0

def test_no_distance_surcharge_at_10():
    """TC09: ровно 10 км - доплаты нет."""
    assert calculate_rental_cost(30, 10, False) == 100.0

# Негативные тесты (TC10 - TC11)

def test_negative_minutes():
    """TC10: отрицательное время - ValueError."""
```

```

with pytest.raises(ValueError):
    calculate_rental_cost(-1, 5, False)

def test_negative_distance():
    """TC11: отрицательное расстояние — ValueError."""
    with pytest.raises(ValueError):
        calculate_rental_cost(10, -1, False)

# Скидка (TC12 - TC13)

def test_student_discount():
    """TC12: скидка студента 30% (базовая)."""
    assert calculate_rental_cost(30, 5, True) == 70.0

def test_student_discount_with_surcharge():
    """TC13: скидка студента + доплата за километры."""
    # база 250 + 1*8 = 258; доплата (15-10)*20 = 100; итого 358; скидка
    358*0.7 = 250.6
    assert calculate_rental_cost(61, 15, True) == pytest.approx(250.6)

# Бесплатная поездка (TC14 - TC15)

def test_zero_minutes():
    """TC14: нулевая поездка — стоимость 0."""
    assert calculate_rental_cost(0, 5, False) == 0.0

def test_zero_minutes_with_student():
    """TC15: нулевая поездка со скидкой — всё равно 0."""
    assert calculate_rental_cost(0, 5, True) == 0.0

```

Задание 5:

Запустил тесты командой `pytest -v --cov=rental --cov-report=term-missing` и получил результаты:

```

(.venv) PS C:\Users\oxana\PycharmProjects\Software_system_testing> pytest -v --cov=rental --cov-report=term-missing
===== test session starts =====
platform win32 -- Python 3.12.2, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\oxana\PycharmProjects\Software_system_testing\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\oxana\PycharmProjects\Software_system_testing
plugins: cov-7.1.0
collected 15 items

test_rental.py::test_minimal Ride PASSED [ 6%]
test_rental.py::test_base_tariff_middle PASSED [ 13%]
test_rental.py::test_second_tariff_middle PASSED [ 20%]
test_rental.py::test_distance_surcharge PASSED [ 26%]
test_rental.py::test_base_tariff_upper_bound PASSED [ 33%]
test_rental.py::test_second_tariff_start PASSED [ 40%]
test_rental.py::test_second_tariff_upper_bound PASSED [ 46%]
test_rental.py::test_third_tariff_start PASSED [ 53%]
test_rental.py::test_no_distance_surcharge_at_10 PASSED [ 60%]
test_rental.py::test_negative_minutes PASSED [ 66%]
test_rental.py::test_negative_distance PASSED [ 73%]
test_rental.py::test_student_discount PASSED [ 80%]
test_rental.py::test_student_discount_with_surcharge PASSED [ 86%]
test_rental.py::test_zero_minutes PASSED [ 93%]
test_rental.py::test_zero_minutes_with_student PASSED [100%]

===== tests coverage =====
coverage: platform win32, python 3.12.2-final-0

Name      tests  Miss  Cover  Missing
-----
rental.py   17      0  100%
TOTAL      17      0  100%

===== 15 passed in 0.02s =====
(.venv) PS C:\Users\oxana\PycharmProjects\Software_system_testing>

```

Рис.1. Результат запуска автотестов

Анализ результата:

Собрано тестов (collected): 15

Пройдено (passed): 15

Провалено (failed): 0

Пропущено (skipped): 0

Код возврата: 0 (успех)

Время выполнения: 0.02 s

Окружение: Python 3.12.2, pytest 9.1.1, плагин cov 7.1.0

Задание 6:

Вывод:

1). Покрытые ветви кода. По результатам запуска (15 passed) и отчёта о покрытии (rental.py — 17 stmts, 0 miss, 100%) исполнены все ветви функции calculate_rental_cost.

№	Ветвь кода	Условие	Покрывающий тест
1	Raise ValueError (время)	minutes < 0	test_negative_minutes
2	raise ValueError (расстояние)	distance_km < 0	test_negative_distance
3	base = 0.0	minutes == 0	test_zero_minutes, test_zero_minutes_with_student
4	base = 100.0	0 < minutes ≤ 30	test_minimal_ride, test_base_tariff_middle, test_base_tariff_upper_bound, test_no_distance_surcharge_at_10, test_student_discount
5	base = 100 + (m-30)·5	30 < minutes ≤ 60	test_second_tariff_middle, test_second_tariff_start, test_second_tariff_upper_bound
6	base = 250 + (m-60)·8	minutes > 60	test_third_tariff_start, test_distance_surcharge, test_student_discount_with_surcharge
7	base += (km-10)·20	distance_km > 10	test_distance_surcharge, test_student_discount_with_surcharge
8	base *= 0.7	is_student = True	test_student_discount, test_student_discount_with_surcharge, test_zero_minutes_with_student

Таким образом, покрыты все ветви — обе ветви валидации, ветка нулевой поездки, все три тарифных диапазона, доплата за расстояние и скидка студента. Покрытие = 100% строк и ветвлений.

Оставшиеся риски:

- 1). Нечисловые входы (None, строки):** возникнет `TypeError`, а не ожидаемый `ValueError`. Причина - валидация проверяет только `< 0`.
- 2). Не-bool для `is_student` (1, 0, "yes"):** тихий проход в ветку скидки (в Python `bool` — подкласс `int`). Причина - нет проверки типа.
- 3). Дробное расстояние у порога (10.001 км):** погрешность `float` при `> 10`. Причина - арифметика чисел с плавающей точкой.
- 4). Очень большие значения `minutes/distance_km`:** нереалистичные суммы, потеря точности, переполнение тарифной логики. Причина - нет верхнего предела валидации.
- 5). Изменение бизнес-правил (новые тарифы):** тесты устареют, если правила поменяются, а тесты — нет. Причина - тесты жёстко фиксируют текущие числа.

Таким образом, не обеспечены устойчивость к некорректным типам данных и защита от «мусорных» входов.

Тестирование важно для качества, так как:

- 1. Раннее обнаружение дефектов.** Правило «1:10:100» показывает, что ошибка, найденная на этапе тестирования модуля, обходится в разы дешевле, чем обнаруженная в продакшене.
- 2. Регрессионная защита.** Набор из 15 автотестов фиксирует текущее поведение. Любое изменение тарифов, порогов или скидок мгновенно покажет, отклонилось ли поведение от бизнес-правил.
- 3. «Живая спецификация».** Каждый тест документирует своё бизнес-правило: глядя на `test_second_tariff_start`, сразу понятно, что 31 минута должна стоить 105.
- 4. Объективная метрика.** Покрытие 100% — подтверждение того, что ни одна строка не осталась непроверенной.
- 5. Место в жизненном цикле (Shift Left).** Тесты писались вместе с кодом, на стадии разработки, а не после релиза. Это снижает стоимость и повышает качество.

Вывод: в ходе выполнения лабораторной работы создал модуль `calculate_rental_cost` (расчёт стоимости аренды электросамоката) с ветвящейся логикой. Выписал 8 бизнес-правил (BR1–BR8): валидация входных данных, нулевая поездка, три тарифных диапазона, доплата за километры свыше 10 и скидка 30%. Определил параметры: входные (`minutes`,

distance_km, is_student) выходной (float), допустимые/недопустимые значения, граничные точки (0, 30, 31, 60, 61 мин; 10 км). Составил 15 тест-кейсов по стандартным атрибутам в категориях: позитивные (4), граничные (5), негативные (2), скидка — «premium» (2), бесплатная поездка(2). Написал 15 автотестов на pytest с использованием assert, pytest.raises (негативные) и pytest.approx (точность float). Тесты успешно запущены: все 15 тестов пройдены (15 passed in 0.02s), покрытие rental.py — 100%. Выполнение работы подтвердило, что тестирование — это важная деятельность жизненного цикла: оно превращает неформальные бизнес-правила в проверяемые утверждения, защищает код от регрессий и даёт объективную картину готовности продукта.